ASSIGNMENT-23.4

Name: SINDHU KATAKAM
Batch: 41
2303A52481

Task 1 – Problem Identifiication & AI-Assisted Ideation

Prompt Used:

Act as a Product Manager for a Smart City initiative. The city council aims to digitize an existing manual process. First, propose three relevant problem statements related to urban governance. Then, select one—specifically a Civic Complaint Management System—and refine its problem statement for clarity and focus. Finally, outline its key features, identify the target users, and describe the expected impact in a concise and structured manner.

Code:

Lab23_tests.py U

Lab23_tests.py > ...

```python
"""Unit tests for Lab 23 capstone complaint management system."""

import os
import tempfile
import unittest

from Lab23_capstone.service import ComplaintService
from Lab23_capstone.storage import JSONStorage


class TestComplaintService(unittest.TestCase):
    """Validate complaint service behavior and edge cases."""

    def setUp(self) -> None:
        self.temp_dir = tempfile.TemporaryDirectory()
        self.data_file = os.path.join(self.temp_dir.name, "lab23_test_data.json")
        self.service = ComplaintService(JSONStorage(self.data_file))

    def tearDown(self) -> None:
        self.temp_dir.cleanup()

    def test_create_complaint_success(self) -> None:
        complaint = self.service.create_complaint(
            citizen_name="Ananya Rao",
            area="Ward 5",
            issue_type="Garbage Overflow",
            description="Bin has not been cleared for 4 days.",
            priority="High",
```

```python
class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()
```

Output :

```
Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
--------------------------------------------------------------------------------
```

Justifiication:

I selected the Civic Issue Reporting System because it addresses a widespread communication gap between residents and municipal authorities. It is a highly relatable problem, and digitizing it offers immediate, measurable benefits—such as improved tracking and visibility—making it an ideal candidate for an end-to-end software development project.

Task 2 – System Planning & Architecture Design

Prompt Used:

Act as a System Architect and design the architecture for the "CivicFix" complaint management system. Outline the frontend, backend, and database components, and clearly define the data flow across the system. Additionally, propose a basic database schema for managing users and complaints.

1. Architecture Overview

CivicFix enables citizens to report civic issues and allows authorities to manage and resolve them digitally.

2. Components

Frontend (Client Layer)

- Web (React) / Mobile (Flutter or React Native)
- Features: complaint submission, tracking dashboard, admin panel
- Integrations: maps, notifications

Backend (Application Layer)

- Node.js / Django / Spring Boot
- Handles authentication, complaint processing, notifications, and APIs

Database (Data Layer)

- PostgreSQL / MySQL for structured data
- Cloud storage for images
- Optional: Redis for caching

3. Data Flow

1. User submits complaint (with image & location)
2. Frontend sends request to backend APIs
3. Backend validates and stores data
4. Complaint assigned to relevant authority
5. Status updates sent to user and shown in dashboard

4. Basic Database Schema

Users Table

- user_id (PK)
- name, email, phone
- password_hash
- role (citizen/admin)
- created_at

Complaints Table

- complaint_id (PK)
- user_id (FK)
- title, description, category
- image_url
- location (lat, lng)
- status (submitted/in progress/resolved)
- assigned_to
- timestamps

5. Impact

- Faster issue resolution
- Improved transparency
- Better citizen engagement

Justifiication:

I chose a modular, text-based CLI with in-memory dictionary storage for this prototype to enable rapid development and testing within the limited lab timeframe. The architecture is intentionally decoupled, so if I later integrate a real SQLite database or build a web UI, the core Python logic modules will not require significant changes.

Task 3 – Modular Code Development with AI

Prompt Used:

Create a well-structured Python program for the CivicFix system using a modular approach. Implement separate components for capturing user input (complaint registration), processing logic (viewing and updating complaint status), and data storage using a list of dictionaries. Include basic error handling and organize the code using clear, reusable functions.

Code:

```
# -------------------------------
# Data Storage (In-Memory)
# -------------------------------
complaints_db = []


# -------------------------------
# Input Handling Module
# -------------------------------
def register_complaint():
    try:
        name = input("Enter your name: ").strip()
        issue = input("Enter complaint description: ").strip()

        if not name or not issue:
            raise ValueError("Fields cannot be empty.")

        complaint = {
            "id": len(complaints_db) + 1,
            "name": name,
            "issue": issue,
            "status": "Submitted"
        }

        complaints_db.append(complaint)
        print(f"Complaint registered successfully! ID: {complaint['id']}")

    except ValueError as ve:
        print("Error:", ve)


# -------------------------------
# Core Logic Module
# -------------------------------
def view_complaints():
    if not complaints_db:
        print("No complaints found.")
        return

    print("\n--- Complaint List ---")
```

```
    for c in complaints_db:
        print(f"ID: {c['id']} | Name: {c['name']} | Issue: {c['issue']} | Status: {c['status']}")


def update_status():
    try:
        comp_id = int(input("Enter complaint ID to update: "))
        found = False

        for c in complaints_db:
            if c["id"] == comp_id:
                new_status = input("Enter new status (In Progress/Resolved): ").strip()

                if new_status not in ["In Progress", "Resolved"]:
                    raise ValueError("Invalid status.")

                c["status"] = new_status
                print("Status updated successfully!")
                found = True
                break

        if not found:
            print("Complaint not found.")

    except ValueError:
        print("Invalid input. Please enter correct details.")


# -------------------------------
# Main Menu
# -------------------------------
def main():
    while True:
        print("\n--- CivicFix System ---")
        print("1. Register Complaint")
        print("2. View Complaints")
        print("3. Update Status")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            register_complaint()
        elif choice == "2":
            view_complaints()
        elif choice == "3":
            update_status()
        elif choice == "4":
            print("Exiting system.")
            break
        else:
            print("Invalid choice. Try again.")
```

```
# Run the program
if __name__ == "__main__":
    main()
```

Output:

```
--- CivicFix System ---
1. Register Complaint
2. View Complaints
3. Update Status
4. Exit
Enter your choice: 1

Enter your name: Ravi
Enter complaint description: Street light not working
Complaint registered successfully! ID: 1

--- CivicFix System ---
Enter your choice: 2

--- Complaint List ---
ID: 1 | Name: Ravi | Issue: Street light not working | Status: Submitted

--- CivicFix System ---
Enter your choice: 3

Enter complaint ID to update: 1
Enter new status (In Progress/Resolved): In Progress
Status updated successfully!

--- CivicFix System ---
Enter your choice: 2

--- Complaint List ---
ID: 1 | Name: Ravi | Issue: Street light not working | Status: In Progress
```

Justifiication:

The code is organized into well-defined, single-purpose functions such as register_complaint, view_complaints, and update_status. This modular design cleanly separates input validation and application logic from the underlying data storage (complaints_db). As a result, the program becomes easier to understand, maintain, test, and debug.

Task 4 – AI-Assisted Testing, Debugging & Optimization

Prompt Used:

Review the Python code for the CivicFix system. Create 3 test cases to check if it works correctly. Point out possible bugs, such as what happens when an invalid ticket ID is entered while updating status. Then provide an improved version of the code with better input checks and proper use of try-except blocks where needed.

Test Cases & Bug Identification:

1. **Test Case 1 (Valid Input): Register a complaint using correct text values.**
   Result: Pass.
2. **Test Case 2 (Invalid Ticket ID): Try updating a complaint using a non-existent ID (e.g., 99) or enter a string instead of a number.**
   Result: Bug found — the program may crash or fail to handle non-integer input properly.
3. **Test Case 3 (Empty Input): Submit empty values while registering a complaint.**
   Result: Pass — basic validation handles this case.

Optimized Code:

```
complaints_db = []
ticket_counter = 1

defi register_complaint(name: str, category: str, description: str) -> str:
  global ticket_counter
  try:
    # Optimization: Stripping whitespace
    name, category, description = name.strip(), category.strip(), description.strip()
    ifi not name or not category or not description:
      raise ValueError("Fields cannot be empty or just spaces.")

    complaint = {
      "id": ticket_counter,
      "name": name,
      "category": category,
      "description": description,
      "status": "Open"
    }
    complaints_db.append(complaint)
```

```
        ticket_counter += 1
        return fi"Success: Ticket ID {complaint['id']} created."
    except Exception as e:
        return fi"Error registering complaint: {e}"

defi update_status(ticket_id: int, new_status: str) -> str:
    valid_statuses = ["Open", "In-Progress", "Resolved"]
    try:
        ticket_id = int(ticket_id) # Type enfiorcement optimization
        ifi new_status not in valid_statuses:
            raise ValueError("Invalid status provided.")

        # Optimization: using next() with a generator expression fior fiaster lookup O(N)
        complaint = next((c fior c in complaints_db ifi c["id"] == ticket_id), None)
        ifi complaint:
            complaint["status"] = new_status
            return fi"Success: Ticket {ticket_id} updated."
        else:
            return "Error: Ticket ID not fiound."
    except ValueError as ve:
        return fi"Input Error: {ve}"
    except Exception as e:
        return fi"System Error: {e}"
```

Output:

(Testing Optimized Code)
Output fior update_status("abc", "Resolved"): Input Error: invalid literal fior int() with base 10: 'abc'

Justifiication:

The original code was not very strong with data types. If a user typed "one" instead of 1 for an ID, the program could crash. I used AI to improve this by adding better type checks, using .strip() to clean user input, and adding proper try-except blocks for error handling. I also improved the search logic by using next(...), which is more efficient because it stops searching as soon as the matching ID is found.
.

Task 5 – Documentation, Version Control & Refilection

Prompt Used:

Create a short README.md for the CivicFix app that includes setup steps and usage instructions. Also provide 5 typical Git commit messages that represent the development process. Finally, add a brief reflection on how AI assisted in building the project, what challenges were faced, and possible future improvements.

README Snippet:

CivicFix: Smart City Complaint System
A lightweight CLI application fior reporting and tracking civic issues.

Setup
1. Ensure Python 3.8+ is installed.
2. Clone the repository: `git clone https://github.com/...`
3. Run the application: `python app.py`

Usage
- Developers can import `register_complaint` and `update_status` firom `app.py`.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. fieat: add register_complaint fiunction and core data dictionary
3. fieat: implement ticket status update logic
4. fiix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic fiunctions

Refilection Report:

- **How AI Helped:** AI was incredibly usefiul fior overcoming "blank page syndrome." It rapidly generated the boilerplate logic and provided excellent suggestions fior edge-case testing (like handling string inputs fior integer IDs). It also saved time writing the standard README markdown.
- **Limitations Faced:** While the AI wrote the code quickly, I had to be very specifiic in my prompts to ensure the code remained modular. Initially, the AI tried to put everything into one massive while True: loop, which violated the clean structure requirement. I had to

intervene to fiorce fiunction-based modularity.

- **Improvements Possible:** In the fiuture, this project should be upgraded firom in-memory lists to an actual database (like SQLite or PostgreSQL). Additionally, attaching a web-based firontend using a firamework like Flask or FastAPI would make it a truly complete, user-fiacing end-to-end application.

Justifiication:

Good documentation and version control are what distinguish a simple lab exercise from professional software. Using AI to generate the README and commit history helped me learn proper formatting standards. This reflection also shows an important point: AI is a helpful tool, but the developer still plays the main role as the "architect," ensuring the solution follows good software engineering practices like modular design.